

Report HW 7 ECE-111

Name: Jasper Huang

PID: A17796149

Homework 7a

SystemVerilog Design Code Snapshot – uart_rx

```
RX_DATA_BIT0: begin
    // sample start bit value at mid-point
    // for each databit to get midpoint count value is 16
    // counting starts from midpoint of previous bit and ends at midpoint
    // of current data bit
    // Student to fill rest of the code
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[0] <= rx;
        count <= 0;
        state <= RX_DATA_BIT1;
    end else begin
        count <= count + 1;
    end
end
```

```
RX_DATA_BIT1: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[1] <= rx;
        count <= 0;
        state <= RX_DATA_BIT2;
    end else begin
        count <= count + 1;
    end
end

RX_DATA_BIT2: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[2] <= rx;
        count <= 0;
        state <= RX_DATA_BIT3;
    end else begin
        count <= count + 1;
    end
end

RX_DATA_BIT3: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[3] <= rx;
        count <= 0;
        state <= RX_DATA_BIT4;
    end else begin
        count <= count + 1;
    end
end

RX_DATA_BIT4: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[4] <= rx;
        count <= 0;
        state <= RX_DATA_BIT5;
    end else begin
        count <= count + 1;
    end
end

RX_DATA_BIT5: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[5] <= rx;
        count <= 0;
        state <= RX_DATA_BIT6;
    end else begin
        count <= count + 1;
    end
end

RX_DATA_BIT6: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[6] <= rx;
        count <= 0;
        state <= RX_DATA_BIT7;
    end else begin
        count <= count + 1;
    end
end

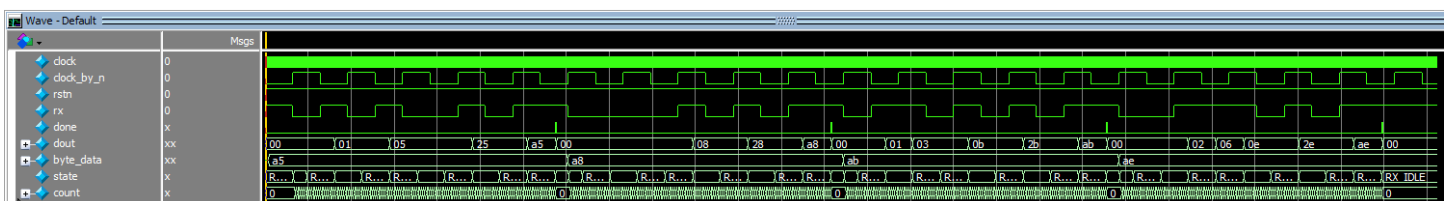
RX_DATA_BIT7: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 0;
        dout[7] <= rx;
        count <= 0;
        state <= RX_STOP_BIT;
    end else begin
        count <= count + 1;
    end
end

RX_STOP_BIT: begin
    if (count == (NUM_CLKS_PER_BIT-1)) begin
        done <= 1;
        count <= 0;
        state <= RX_IDLE;
    end else begin
        count <= count + 1;
    end
end
```

Synthesis Resource Usage – uart_rx

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	24
2		
3	✓ Combinational ALUT usage for logic	32
1	-- 7 input functions	1
2	-- 6 input functions	14
3	-- 5 input functions	4
4	-- 4 input functions	9
5	-- <=3 input functions	4
4		
5	Dedicated logic registers	24
6		
7	I/O pins	12
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clk~input
12	Maximum fan-out	24
13	Total fan-out	237
14	Average fan-out	2.96

Simulation Snapshot and Result Explanation – uart_rx



This simulation wave shows the rx receiving the data from rx and changes state between reading each bit from state RX_DATA_BIT0 to RX_DATA_BIT7 to read each bit and combine to dout which then high the done flag to indicate that the data output is ready. The data output a5 a8 ab ae are all correct output based on the input rx.

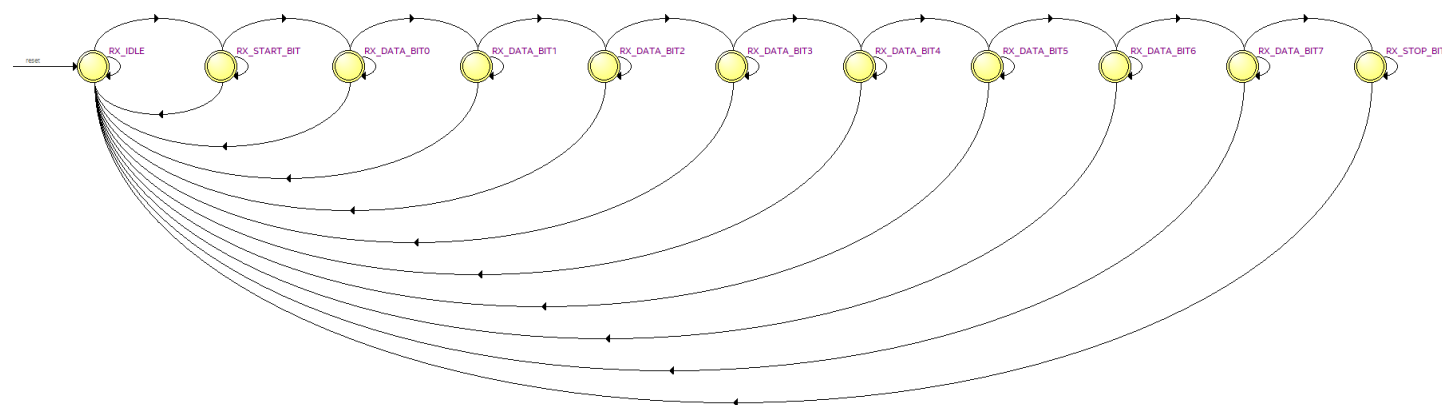
Modelsim Transcript Window – uart_rx

```

VSIM 22> run -all
# Test Passed - Correct Byte Received time=          3200  expected=a5  actual=a5
# Test Passed - Correct Byte Received time=          6400  expected=a8  actual=a8
# Test Passed - Correct Byte Received time=          9600  expected=ab  actual=ab
# Test Passed - Correct Byte Received time=         12800  expected=ae  actual=ae
# ** Note: $finish      : C:/Users/Jaspe/repositories/ECE 111 Lab HW/Homework7/Lab7/uart_top/uart_rx/uart_rx_testbench.sv(91)
# Time: 13620 ns  Iteration: 0  Instance: /uart_rx_testbench

```

FSM State Transition Diagram and Table – uart_rx



	Source State	Destination State	Condition
3	RX_DATA_BIT0	RX_DATA_BIT1	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
4	RX_DATA_BIT1	RX_IDLE	(!rstn)
5	RX_DATA_BIT1	RX_DATA_BIT2	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
6	RX_DATA_BIT1	RX_DATA_BIT1	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
7	RX_DATA_BIT2	RX_IDLE	(!rstn)
8	RX_DATA_BIT2	RX_DATA_BIT3	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
9	RX_DATA_BIT2	RX_DATA_BIT2	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
10	RX_DATA_BIT3	RX_DATA_BIT4	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
11	RX_DATA_BIT3	RX_IDLE	(!rstn)
12	RX_DATA_BIT3	RX_DATA_BIT3	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
13	RX_DATA_BIT4	RX_DATA_BIT4	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
14	RX_DATA_BIT4	RX_DATA_BIT5	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
15	RX_DATA_BIT4	RX_IDLE	(!rstn)
16	RX_DATA_BIT5	RX_DATA_BIT5	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
17	RX_DATA_BIT5	RX_IDLE	(!rstn)
18	RX_DATA_BIT5	RX_DATA_BIT6	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
19	RX_DATA_BIT6	RX_IDLE	(!rstn)
20	RX_DATA_BIT6	RX_DATA_BIT7	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
21	RX_DATA_BIT6	RX_DATA_BIT6	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
22	RX_DATA_BIT7	RX_STOP_BIT	(count[0]).(count[1]).(count[2]).(count[3]).(rstn)
23	RX_DATA_BIT7	RX_IDLE	(!rstn)
24	RX_DATA_BIT7	RX_DATA_BIT7	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
25	RX_IDLE	RX_START_BIT	(!rx).(rstn)
26	RX_IDLE	RX_IDLE	(!rx).(rstn) + (rx)
27	RX_START_BIT	RX_START_BIT	(!always0).(rstn)
28	RX_START_BIT	RX_IDLE	(!rstn)
29	RX_START_BIT	RX_DATA_BIT0	(always0).(rstn)
30	RX_STOP_BIT	RX_STOP_BIT	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn)
31	RX_STOP_BIT	RX_IDLE	(!count[0]).(rstn) + (count[0]).(!count[1]).(rstn) + (count[0]).(count[1]).(!count[2]).(rstn) + (count[0]).(count[1]).(count[2]).(!count[3]).(rstn) + (count[0]).(count[1]).(count[2]).(count[3])

Homework 7b

SystemVerilog Design Code Snapshot – uart_top

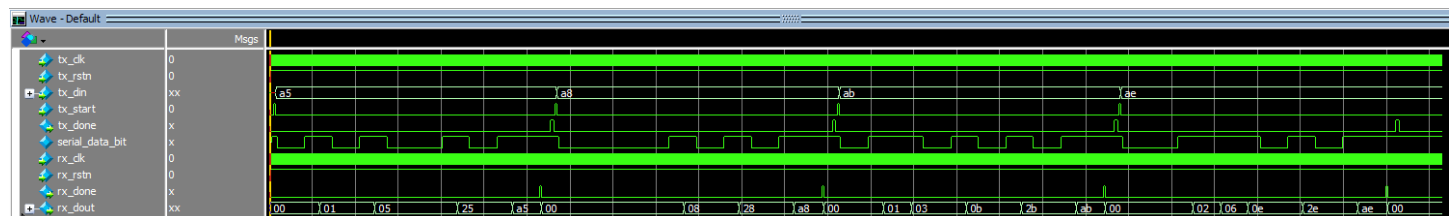
```
// Instantiate uart transmitter module
// student to add code
uart_tx #(.NUM_CLKS_PER_BIT(NUM_CLKS_PER_BIT)) tx(
    .clk(tx_clk),
    .rstn(tx_rstn),
    .din(tx_din),
    .start(tx_start),
    .done(tx_done),
    .tx(serial_data_bit)
);

// Instantiate uart receiver module
// student to add code
uart_rx #(.NUM_CLKS_PER_BIT(NUM_CLKS_PER_BIT)) rx(
    .clk(rx_clk),
    .rstn(rx_rstn),
    .rx(serial_data_bit),
    .done(rx_done),
    .dout(rx_dout)
);
```

Synthesis Resource Usage – uart_top

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	38
2		
3	▼ Combinational ALUT usage for logic	51
1	-- 7 input functions	3
2	-- 6 input functions	21
3	-- 5 input functions	9
4	-- 4 input functions	12
5	-- <=3 input functions	6
4		
5	Dedicated logic registers	38
6		
7	I/O pins	23
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	rx_clk~input
12	Maximum fan-out	24
13	Total fan-out	379
14	Average fan-out	2.81

Simulation Snapshot and Result Explanation – uart_top



The simulation wave shows the data transmitted from tx to rx for four different values. Each of the four values are first updated in din and start the tx. The data is then transmitted through the serial_data_bit to rx and then the rx_done flag is up once the dout is ready. From the wave, all data is transmitted correctly from tx to rx.

Modelsim Transcript Window – uart_top

```

VSIM 51> run -all
# Test Passed - Correct Byte Received time=          3150  expected=a5  actual=a5
# Test Passed - Correct Byte Received time=          6430  expected=a8  actual=a8
# Test Passed - Correct Byte Received time=          9710  expected=ab  actual=ab
# Test Passed - Correct Byte Received time=         12990  expected=ae  actual=ae
# ** Note: $finish      : C:/Users/Jaspe/repositories/ECE 111 Lab HW/Homework7/Lab7/uart_top/uart_top_testbench.sv(109)
#   Time: 13650 ns  Iteration: 0  Instance: /uart_top_testbench

```

Homework 7c

SystemVerilog Design Code Snapshot – uart_control_system

```
// Instantiate UART TX CONTROL Module
uart_tx_control #(.NUM_OF_BYTES(NUM_OF_BYTES)) tx_control_fsm(
    .clk(clock),
    .rstn(rstn),
    .mem_read_data(mem_read_data), // connect to mem_read_data input primary port
    .mem_read_addr(mem_read_addr), // connect to mem_read_addr output primary port
    .mem_read_enable(mem_read_enable), // connect to mem_read_enable output primary port
    .transmission_done(transmission_done), // connect to transmission_done output primary port
    .uart_tx_done(tx_done), // connect to tx_done coming from uart_top module instance
    .uart_tx_data(tx_data), // connect to tx_data going into uart_top module instance
    .uart_tx_start(tx_start) // connect to tx_start going into uart_top module instance
);

// Instantiate UART RX CONTROL Module
// Note : Student to make connections below for uart_rx_control module instantiation
uart_rx_control #(.NUM_OF_BYTES(NUM_OF_BYTES)) rx_control_fsm(
    .clk(clock),
    .rstn(rstn),
    .mem_write_data(mem_write_data), // connect to mem_write_data output primary port
    .mem_write_addr(mem_write_addr), // connect to mem_write_addr output primary port
    .mem_write_enable(mem_write_enable), // connect to mem_write_enable output primary port
    .message_received(message_received), // connect to message_received output primary port
    .uart_rx_done(rx_done), // connect to rx_done coming from uart_top module instance
    .uart_rx_data(rx_data) // connect to rx_data coming from uart_top module instance
);

// Instantiate UART TOP Module
// uart_top module code has two child modules instantiated : uart_rx and uart_tx modules
// and uart_tx outout tx signal is connected to input rx signal of uart_rx module
// See definition of uart_top in uart_top.sv
uart_top #(.NUM_CLKS_PER_BIT(NUM_CLKS_PER_BIT)) uart_top_inst(
    .tx_clk(clock),
    .tx_rstn(rstn),
    .rx_clk(clock),
    .rx_rstn(rstn),
    .tx_start(tx_start), // connected to uart_tx_start port of uart_tx_control module instance
    .tx_done(tx_done), // connected to uart_tx_done port of uart_tx_control module instance
    .tx_din(tx_data), // connected to uart_tx_data port of uart_tx_control module instance
    .rx_done(rx_done), // connected to uart_rx_done port of uart_rx_control module instance
    .rx_dout(rx_data) // connected to uart_rx_data port of uart_rx_control module instance
);
```

SystemVerilog Design Code Snapshot – rx_control

```
// local variable
logic [7:0] received_data;

// Variable to count number of data bytes received
integer j;

// state encoding and state variable
enum logic[1:0]{
    IDLE      = 2'b00, // IDLE FSM state
    WAIT      = 2'b01, // FSM state to wait for uart_rx FSM to send data byte to uart_rx_control FSM
    WRITE     = 2'b10 // FSM state to write data byte to write to RAM memory in testbench
} state;

// FSM with single always block for next state,
// present state flipflop and output logic
// Note : use non-blocking assignment statement in always_ff block.
// Do not have any blocking assignment statements inside always_ff block
always_ff@(posedge clk) begin
    if(!rstn) begin
        mem_write_addr <= 0;
        mem_write_data <= 0;
        mem_write_enable <= 0;
        message_received <= 0;
        j <= 0;
        state <= IDLE;
    end
    else begin
        case(state)
            // Initialize memory write address, write enable control and memory write data signals to 0
            // Initialize message_received, j to 0
            // Then move to WAIT state
            IDLE: begin
                mem_write_addr <= 0;
                mem_write_data <= 0;
                mem_write_enable <= 0;
                message_received <= 0;
                j <= 0;
                state <= WAIT;
            end
        end
    end
end
```

```
// Wait for uart_rx FSM to indicate data byte is available
// This is done by waiting for uart_rx_done == 1 and then read uart_rx_data sent by uart_rx FSM
// and store it to received_data local variable. Then move to WRITE state to write data byte
// to RAM memory in testbench
// Check if all data bytes have been received from uart_rx FSM, if yes then move to WAIT state otherwise
// wait for uart_rx_done == 1 as mentioned above.
WAIT: begin
    if(j < NUM_OF_BYTES) begin
        if(uart_rx_done == 1) begin
            // Student to add code here
            received_data <= uart_rx_data;
            state <= WRITE;
        end
        else begin
            // Student to add code here
            state <= WAIT;
        end
    end
    else begin
        // Student to add code here
        state <= IDLE;
        message_received <= 1;
    end
end

// Write data byte to RAM memory inside testbench
// This can be achieved by setting mem_write_enable to 1, set mem_write_addr to 'j' to increment
// memory address and copy received_data to mem_write_data
WRITE: begin
    // Note : Do not have mem_write_addr = mem_write_addr + 1 instead assign j to mem_write_addr as shown below
    mem_write_addr <= j;
    // Student to add code here
    mem_write_enable <= 1;
    mem_write_data <= received_data;
    j <= j + 1;
    state <= WAIT;
end

// In Default state move to IDLE state
default: begin
    state <= IDLE;
end
```

SystemVerilog Design Code Snapshot – tx_control

```
// FSM with single always block for next state,
// present state flipflop and output logic
// Note : use non-blocking assignment statement in always_ff block.
// Do not have any blocking assignment statements inside always_ff block
always_ff@(posedge clk) begin
    if(!rstn) begin
        mem_read_addr <= 0;
        mem_read_enable <= 0;
        transmission_done <= 0;
        uart_tx_data <= 0;
        uart_tx_start <= 0;
        state <= IDLE;
        j <= 0;
    end
    else begin
        case(state)
            // Initialize memory read address, read enable control signals to 0
            // Initialize transmission_done, uart_tx_data, uart_tx_start, j to 0
            // Then move to READ state
            IDLE: begin
                mem_read_addr <= 0;
                mem_read_enable <= 0;
                transmission_done <= 0;
                uart_tx_data <= 0;
                uart_tx_start <= 0;
                state <= READ;
                j <= 0;
            end
            // Read all data bytes from ROM model which is instantiated in testbench
            // To achieve this set mem_read_data to '1' and set mem_read_addr to value 'j'
            // j is incrementing memory address value it should be set in READ state
            // Check if all 'j' count is less than NUM_OF_BYTES. i.e. if all data bytes from ROM
            // in testbench has been transmitted by uart_tx to uart_rx
            // If all data bytes transmitted then move to IDLE state and reset mem_read_enable to 0
            // otherwise read next data byte from ROM memory in testbench and then move to DELAY state
            READ: begin
                if(j < NUM_OF_BYTES) begin
                    // Student to add code here
                    // Note : Do not have mem_read_addr = mem_read_addr + 1 instead assign j to mem_read_addr
                    // as 'j' has been already incremented in wait state and new value is available to indicate next read address
                    mem_read_addr <= j;
                    mem_read_enable <= 1;
                    state <= DELAY;
                end
                else begin
                    // Student to add code here
                    state <= IDLE;
                    mem_read_enable <= 0;
                end
            end
            // Important : ROM/RAM memory will take 1 cycle to provide data on mem_read_data
            // after mem_read_enable is set to '1'
            // mem_read_enable is set to '1' in READ FSM, so this DELAY state is required
            // before read data byte is further sent by uart_tx_control FSM to uart_tx input din port
            // Then move to TRANSMIT state
            DELAY: begin
                state <= TRANSMIT;
            end
            // Indicate uart_tx FSM that din data byte is available for transmission and
            // uart_tx should start serial transmission each bit in data byte to uart_rx
            // To do this, set uart_tx_start to '1' and pass mem_read_data to uart_tx_data
            // Then move to WAIT state
            TRANSMIT: begin
                // Student to add code here
                uart_tx_data <= mem_read_data;
                uart_tx_start <= 1;
                state <= WAIT;
            end
        end
    end
end

// Wait until uart_tx has completed serial transmission of data byte to uart_rx
// To achieve this wait for uart_tx_done == 1 which is coming from uart_tx to uart_tx_control FSM
// And then increment 'j' by '1' to indicate 1 data byte has been sent to uart_rx
// Then move to READ state if uart_tx_done is '1' otherwise state in WAIT state
// until uart_tx_done from uart_tx FSM is '1'
WAIT: begin
    if(uart_tx_done == 1) begin
        // Student to add code here
        // Remember to increment j <= j + 1 here
        j <= j + 1;
        state <= READ;
    end
    else begin
        // Student to add code here
        state <= WAIT;
    end
end

// In Default state move to IDLE state
default: begin
    state <= IDLE;
end
```

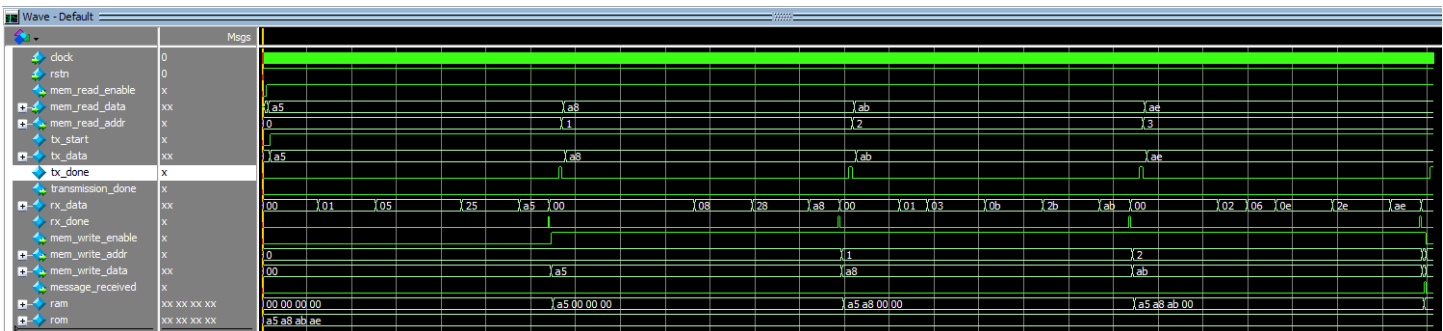
Synthesis Resource Usage – uart_control_system

Analysis & Synthesis Resource Usage Summary

<<Filter>>

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	132
2		
3	Combinational ALUT usage for logic	227
1	-- 7 input functions	3
2	-- 6 input functions	32
3	-- 5 input functions	10
4	-- 4 input functions	95
5	-- <=3 input functions	87
4		
5	Dedicated logic registers	145
6		
7	I/O pins	30
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clock~input
12	Maximum fan-out	145
13	Total fan-out	1298
14	Average fan-out	3.00

Simulation Snapshot and Result Explanation – uart_control_system



From the wave, the data is first read into `mem_read_data`, it's then got the first available data and then transmitted through uart to mem write, which you can see that the rom contains the original information and ram is updated based on the transmitted data through uart and write into the ram. All four data are correctly transmitted.

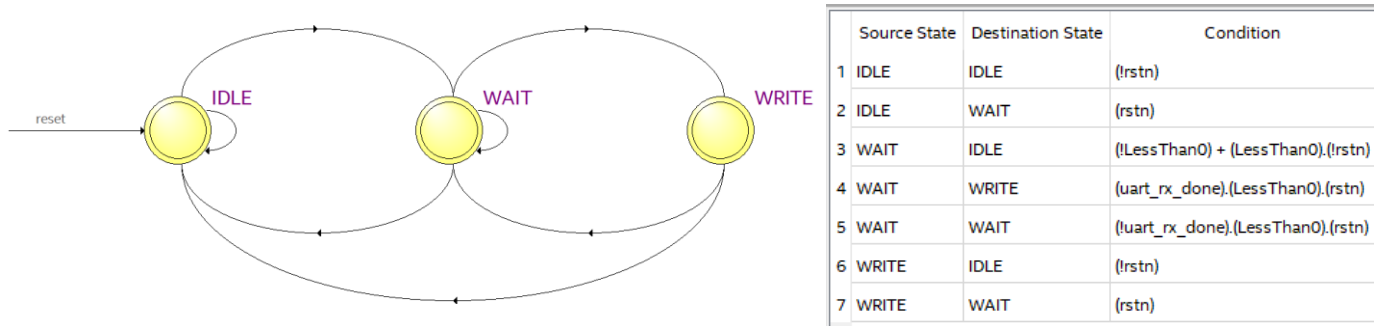
Modelsim Transcript Window – uart_control_system

```

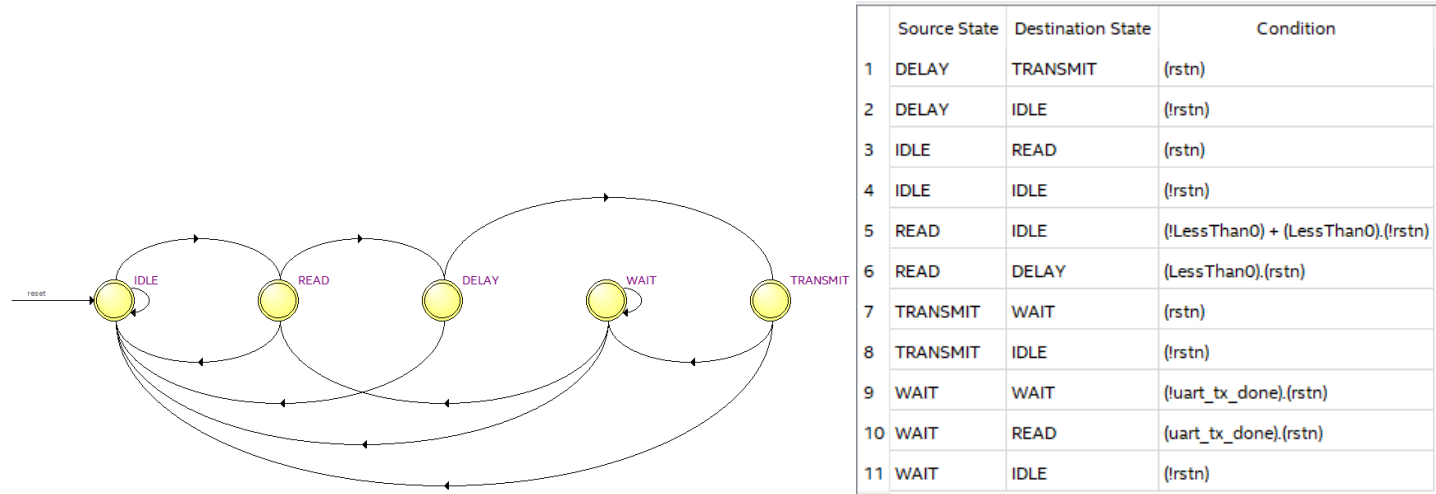
VSIM83> run 14000 ns
# GetModuleFileName: wè'è'w[*@À@èÛ'fB
#
#
# Test Passed - Correct Byte 0 Received time=12980 ns expected byte data=a5 actual byte data=a5
# Test Passed - Correct Byte 1 Received time=12980 ns expected byte data=a8 actual byte data=a8
# Test Passed - Correct Byte 2 Received time=12980 ns expected byte data=ab actual byte data=ab
# Test Passed - Correct Byte 3 Received time=12980 ns expected byte data=ae actual byte data=ae
# ** Note: $finish : C:/Users/Jaspe/repositories/ECE 111 Lab HW/Homework7/Lab7/uart control system/uart control system testbench.sv(134)

```

FSM State Transition Diagram and Table – uart_rx_control



FSM State Transition Diagram and Table – uart_tx_control



Homework 7d

SystemVerilog Design Code Snapshot – vending_machine_moore_reduced_state

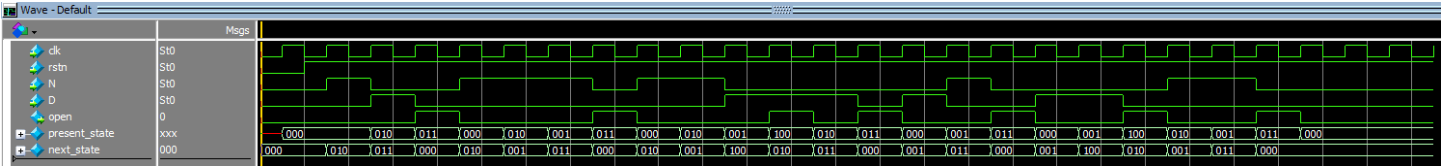
```
parameter[2:0] S1=3'b000, S2=3'b001, S3=3'b010, S4=3'b011, S5=3'b100;
logic[2:0] present_state, next_state;

always_ff@(posedge clk) begin
    if(!rstn) begin
        present_state <= S1;
    end else begin
        present_state <= next_state;
    end
end

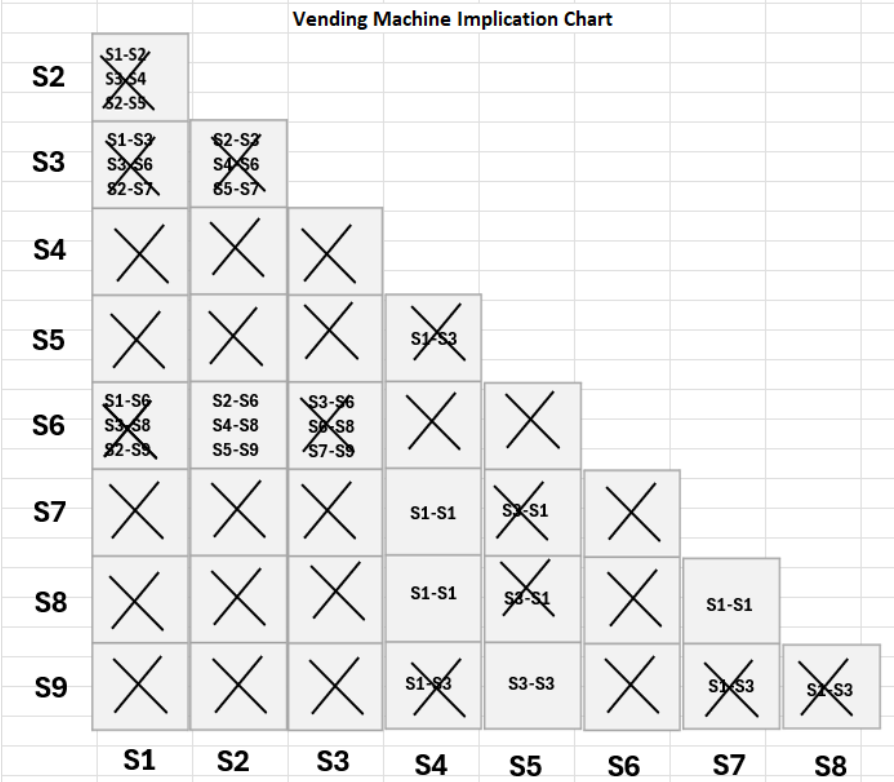
always_comb begin
    next_state = present_state;
    open = 0;

    case (present_state)
        S1: begin
            if (N) next_state = S3;
            else if (D) next_state = S2;
            else next_state = S1;
        end
        S2: begin
            if (N) next_state = S4;
            else if (D) next_state = S5;
            else next_state = S2;
        end
        S3: begin
            if (N) next_state = S2;
            else if (D) next_state = S4;
            else next_state = S3;
        end
        S4: begin
            next_state = S1;
            open = 1;
        end
        S5: begin
            next_state = S3;
            open = 1;
        end
        default: begin
            next_state = S1;
        end
    endcase
end
```

Simulation Waveform Snapshot – vending_machine_moore_reduced_state



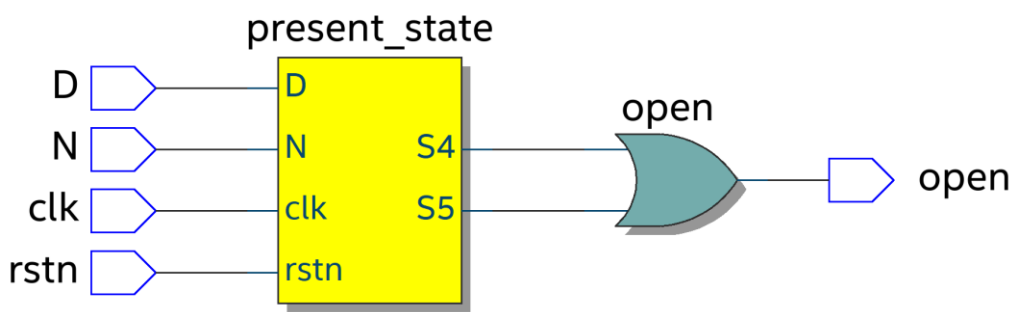
Implication Chart Snapshot – vending_machine_moore_reduced_state



Reduced State Transition Table – vending_machine_moore_reduced_state

Vending Machine State Transition Table after FSM Minimization					
Present State	Next State				Output
	D=0, N=0	D=0, N=1	D=1, N=0	D=1, N=1	
S1	S1	S3	S2	X	0
S2	S2	S4	S5	X	0
S3	S3	S2	S4	X	0
S4	S1	X	X	X	1
S5	S3	X	X	X	1

RTL Post Netlist Viewer – vending_machine_moore_reduced_state



Resource Usage Table– vending_machine_moore_reduced_state

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	4
2		
3	▼ Combinational ALUT usage for logic	6
1	-- 7 input functions	0
2	-- 6 input functions	2
3	-- 5 input functions	2
4	-- 4 input functions	1
5	-- <=3 input functions	1
4		
5	Dedicated logic registers	5
6		
7	I/O pins	5
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	D~input
12	Maximum fan-out	5
13	Total fan-out	44
14	Average fan-out	2.10